PROBE

A self-grading AI incident-response agent

Forge the Future 2026
Elastic + AWS Bedrock Track

WHAT PROBE IS

PROBE is a three-agent system that diagnoses the real root cause of a production incident, not just its symptoms, and proves its own accuracy on stage against a known answer. It watches live telemetry, reasons over the service dependency graph with Claude on Amazon Bedrock, and automatically files a remediation ticket — then grades itself against faults the team deliberately injected.

THE PROBLEM

On-call engineers waste 20–40 minutes per production incident just figuring out which microservice actually broke. When one service fails, a dozen downstream services also start throwing errors, and static threshold alerts can't tell the real cause from the symptoms. Teams end up chasing the loudest alert, not the source.

SOLUTION OVERVIEW

`Detector`	Scans live telemetry in Elasticsearch for statistically significant anomalies across traces, metrics, and logs as an incident unfolds.
`Correlator`	Powered by Claude on Amazon Bedrock. Reasons through the service dependency graph to separate the one anomaly that caused the incident from the many that are just knock-on symptoms.
`Remediator`	Matches the diagnosed cause to a fix in a runbook library and automatically files a ticket, closing the loop from detection to action.

WHY ELASTICSEARCH IS STRUCTURALLY NECESSARY

Root-causing requires correlating traces, metrics, and logs across dozens of services in real time, then reasoning over that correlated view — not just searching it. Elasticsearch is the only piece of the stack that can hold all three telemetry types together, run change-point detection over them natively with ES|QL, and hand a structured, queryable context window to the Correlator agent. A log-only or metrics-only store would leave PROBE guessing.

SELF-GRADING EVALUATION METHODOLOGY

The team deliberately breaks their own test system using feature flags (flagd), so the correct diagnosis is known in advance. PROBE's output is then checked against that ground truth live, on stage, head-to-head against a traditional threshold-alerting baseline. It's not a chat demo — it produces a provable number.

TARGET METRICS

Root-caused 8 of 10 injected faults at a mean 40 seconds; threshold alerts fired on symptoms for only 3 and never named the cause.

TECHNOLOGY STACK

AI / Reasoning — Amazon Bedrock (Claude, ap-south-1 Mumbai)

Search / Data — Elasticsearch, ES|QL, Kibana

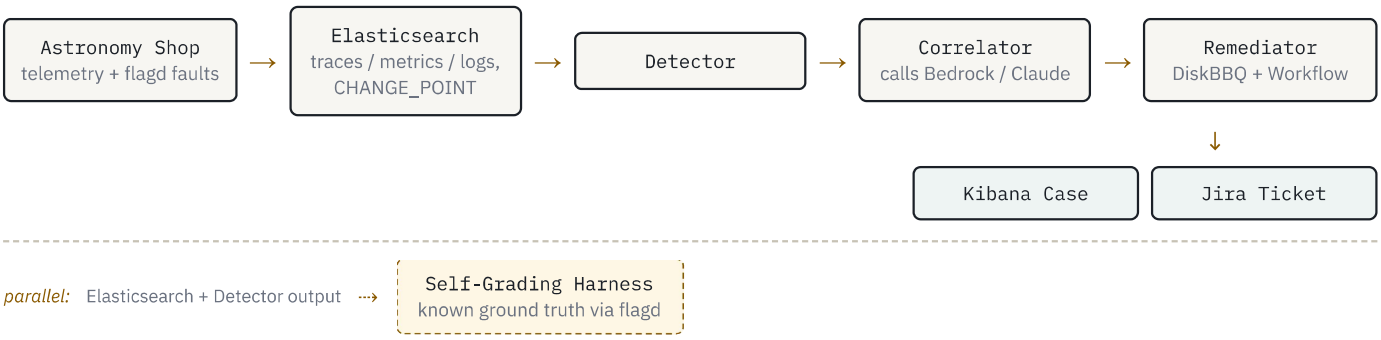
Agents / Orchestration — Elastic Agent Builder, Elastic Workflows

Retrieval — DiskBBQ + Jina AI (vector search)

Telemetry source — OTel Demo "Astronomy Shop" (Elastic EDOT fork)

Infra / Ticketing — Docker, Kubernetes, Jira

ARCHITECTURE DIAGRAM



BUILD PLAN

Day	Focus
Day 0	Pre-staged: cluster + telemetry corpus + Bedrock connector setup.
Day 1	Build the three agents and the self-grading harness.
Day 2	Kibana dashboard build-out and live-demo rehearsal.

BIGGEST RISK

Standing up the demo cluster and generating a clean, labelled telemetry corpus is a multi-day job that must be pre-staged, or the two build days evaporate into infra work. Mitigated by the team's fault-injection strength and rules permitting pre-finale data prep.

MARKET POTENTIAL

Any SRE or platform engineering org is a buyer — this is a category, self-verifying incident-response agents, not a one-off demo.